

SIMO CO2

Sistema de Monitoreo de Huella de Carbono

Bitacora de Desarrollo — Registro de Consultas y Aprendizajes

Registro de las principales consultas realizadas durante el desarrollo del proyecto, junto con la orientacion recibida y los aprendizajes obtenidos en cada etapa.

Proyecto	Proyecto 5 — Sistema de Monitoreo de Huella de Carbono SIMO CO2
Asignatura	Taller de Programacion Aplicada — 3 Semestre
Docente	Victor Arcides Saldivia Vera
Universidad	Universidad de Los Lagos — Depto. Ciencias de la Ingenieria
Stack	TypeScript · React · HTML · CSS · ECharts · jsPDF
Periodo	Abril – Junio 2026

Categorias de entrada:

CONSULTA	PATRON	CODIGO	DISENO	DOCUMENTO	AJUSTE
----------	--------	--------	--------	-----------	--------

1. Registro de Consultas y Aprendizajes

#0
1

CONSULTA

Sesion 1 — Inicio del proyecto

Consulta realizada:

Quiero entender como estructurar un sistema web de monitoreo industrial. Tenemos que implementar patrones de diseno GoF pero no se como organizar los modulos. Como puedo empezar?

Orientacion y resultado:

Se explico la arquitectura de 6 modulos (A al F) basada en el enunciado del proyecto. Cada modulo tiene una responsabilidad clara: emulacion de sensores, distribucion de datos, visualizacion, calculo ambiental, repositorio global y gestion de roles. Se mostro como cada modulo corresponde a un patron GoF especifico.

Aprendizaje clave:

Que la arquitectura modular es la base del diseno orientado a objetos. Separar responsabilidades desde el inicio evita que el codigo se vuelva monolitico y hace mas facil aplicar patrones.

#0
2

PATRON

Sesion 1 — Patrones GoF

Consulta realizada:

Como puedo implementar el patron Observer para que los widgets del dashboard se actualicen solos cuando un sensor genera un nuevo dato? No quiero que el sensor sepa quienes lo escuchan.

Orientacion y resultado:

Se guio la implementacion del Observer con una interface IWidget que define el metodo actualizar(), una clase SensorObserver que mantiene la lista de widgets suscritos, y el metodo notificar() que itera sobre ellos. En React esto se expresa naturalmente como un custom hook useSensorObserver().

Aprendizaje clave:

El patron Observer desacopla completamente la fuente de datos (sensor) de los consumidores (widgets). Agregar un nuevo widget solo requiere implementar la interface y suscribirlo, sin tocar el codigo del sensor.

#0
3

PATRON

Sesion 1 — Patrones GoF

Consulta realizada:

Necesito que el sistema pueda cambiar entre la normativa chilena DS-138 y la europea ISO 14064 para calcular los kg de CO2. Como hago para que esto sea intercambiable sin tocar el resto del código?

Orientacion y resultado:

Se explico el patron Strategy: definir una interface `IEstrategiaCalculo` con el metodo `calcular(tipo, valor)`, luego crear dos clases concretas `EstrategiaChilena` y `EstrategiaEuropea` que la implementan con sus propios factores de conversion. El contexto (`SensorObserver`) usa la estrategia activa sin conocer cual es.

Aprendizaje clave:

Strategy permite cambiar algoritmos en tiempo de ejecucion sin modificar el contexto. Es ideal cuando hay multiples variantes de un mismo calculo que deben ser intercambiables.

#0
4

PATRON

Sesion 1 — Arquitectura

Consulta realizada:

Necesitamos un estado global para el sistema: sensores activos, normativa vigente, historial de datos. Como evito que haya multiples instancias con datos distintos?

Orientacion y resultado:

Se explico el patron Singleton con un IIFE en JavaScript/TypeScript. La clase `AppRepository` tiene un metodo estatico `getInstance()` que crea la instancia solo la primera vez y siempre retorna la misma. Esto garantiza coherencia en todo el sistema.

Aprendizaje clave:

El Singleton garantiza un unico punto de verdad para el estado global. En sistemas de monitoreo esto es critico porque todos los modulos deben leer el mismo historial y la misma configuracion.

#0
5

DOCUMENTO

Sesion 2 — UML

Consulta realizada:

Como puedo crear un diagrama de clases UML que muestre correctamente las relaciones entre mis modulos? No entiendo bien la diferencia entre herencia, composicion y realizacion en UML.

Orientacion y resultado:

Se genero el diagrama UML A3 con la simbologia correcta: herencia (linea solida + triangulo vacio), realizacion de interface (linea discontinua + triangulo vacio), composicion (rombo relleno + linea), asociacion (flecha abierta). Cada modulo quedo enmarcado con su color y etiqueta.

Aprendizaje clave:

La simbologia UML es estandar y el profesor la evalua. Herencia es 'es un', composicion es 'tiene un' (el contenido no puede existir sin el contenedor), y realizacion es 'implementa el contrato de'.

#0 6

DISEÑO

Sesion 3 — CSS y diseño

Consulta realizada:

Cambie el fondo de la pantalla de login a verde pero ahora los inputs y botones no se ven. Como puedo hacer que hagan contraste sin cambiar la paleta de colores del proyecto?

Orientacion y resultado:

Se identificaron dos errores en style.css: la variable --blanco tenia asignado #37474F en vez de #FFFFFF, y el header usaba esa variable quedando verde. Se corrigio asignando blanco real y se agrego un panel semitransparente al formulario para separarlo visualmente del fondo sin tapar el diseño.

Aprendizaje clave:

Las variables CSS mal asignadas son errores silenciosos que afectan todo el sistema visual. Siempre verificar que --blanco sea realmente blanco y que los colores de fondo y texto tengan suficiente contraste (relacion minima 4.5:1 para accesibilidad).

#0 7

CODIGO

Sesion 3 — Dashboard

Consulta realizada:

Quiero que todo el CSS del dashboard quede en el style.css y no dentro del HTML. Como organizo esto para que sea mas limpio y profesional?

Orientacion y resultado:

Se explico como usar variables CSS en :root para centralizar la paleta del dashboard (--db-bg, --db-surface, etc.) y como mover todos los selectores del bloque style interno al archivo CSS externo. El HTML quedo solo con clases semanticas y sin estilos inline.

Aprendizaje clave:

Separar el CSS del HTML es un principio basico de mantenibilidad. Con variables CSS en :root cualquier cambio de color se hace en un solo lugar y se propaga a todo el sistema automaticamente.

#0
8

CODIGO

Sesion 4 — ECharts

Consulta realizada:

Como puedo crear graficos con ECharts que se actualicen en tiempo real? Quiero que el grafico de barras muestre una semana cada 10 segundos y que entre semanas los datos fluctuen visualmente.

Orientacion y resultado:

Se implemento un sistema de doble intervalo: un tick rapido cada 300ms que llama a `generarLectura()` y actualiza el grafico con una barra 'viva' (marcada con triangulo), y un consolidador cada 10 segundos que promedia las lecturas, fija la barra definitiva y agrega la siguiente semana al trimestre.

Aprendizaje clave:

ECharts permite actualizar datos en caliente con `setOption()` sin reiniciar el grafico. La clave es mantener arrays de datos externos y pasarlos a `setOption` en cada actualizacion. Para animacion suave usar `animationDuration` y `animationEasing`.

#0
9

CODIGO

Sesion 4 — jsPDF

Consulta realizada:

Como puedo generar un PDF desde el navegador con los datos del dashboard sin usar un servidor? Necesito que incluya tablas de consumo, CO2 por sensor y los factores de la normativa activa.

Orientacion y resultado:

Se integro la libreria jsPDF via CDN. Se construyo el PDF dibujando rectangulos de color para las secciones, texto con `setFont/setTextColor`, y tablas con `rect()` iterando filas. El PDF incluye header con el nombre del sistema, 3 tarjetas de resumen, tabla por sensor con porcentaje de CO2, detalle semanal y factores de la normativa.

Aprendizaje clave:

jsPDF trabaja con coordenadas en mm desde la esquina superior izquierda. Es recomendable llevar una variable 'y' que va sumando la altura de cada elemento. Agregar `doc.addPage()` cuando y supera ~270 para evitar que el contenido se corte.

#1
0

AJUSTE

Sesion 4 — Simulacion

Consulta realizada:

La simulacion trimestral llega a 12 semanas pero quiero que al completar el trimestre el grafico se reinicie automaticamente y comience el siguiente ciclo. Como manejo esto sin perder el historial?

Orientacion y resultado:

Se implemento un contador semanaEnCiclo separado del contador global semanaTotal. Al llegar a 12 el ciclo se reinicia: los arrays del grafico (labelsEje, dataAgua, etc.) se vacian con length=0, el grafico se limpia con setOption, y cicloNum aumenta. El historial del repositorio conserva todas las semanas con su numero de ciclo.

Aprendizaje clave:

Separar el estado visual (lo que se muestra en el grafico) del estado persistente (el historial completo) es fundamental. El grafico puede reiniciarse sin perder datos porque el repositorio los guarda independientemente.

2. Resumen de Tecnologías y Patrones Aprendidos

Patron / Tecnologia	Modulo	Aplicacion en SIMO CO2
Observer (GoF)	B	SensorObserver notifica a IWidget[] cada 300ms sin acoplamiento directo
Strategy (GoF)	D	IEstrategiaCalculo permite cambiar normativa DS-138 / ISO 14064 en runtime
Singleton (GoF)	E	AppRepository.getInstance() garantiza un unico estado global coherente
Factory (GoF)	A+C	SensorFactory y WidgetFactory desacoplan la creacion de objetos de su uso
TypeScript	-	Interfaces y tipos estaticos que hacen cumplir los contratos de los patrones
ECharts	C	Graficos de barras animados con doble intervalo (tick 300ms + consolidacion 10s)
jsPDF	F	Generacion de informes PDF desde el navegador sin servidor
CSS Variables	-	Paleta centralizada en :root para cambios globales con una sola linea

Conclusion: El desarrollo de SIMO CO2 demostro que los patrones GoF no son teoricos: Observer, Strategy, Singleton y Factory resolvieron problemas concretos de desacoplamiento, intercambiabilidad y coherencia de estado. La clave fue identificar el problema primero y luego elegir el patron que mejor lo resuelve.